

# Perceptron

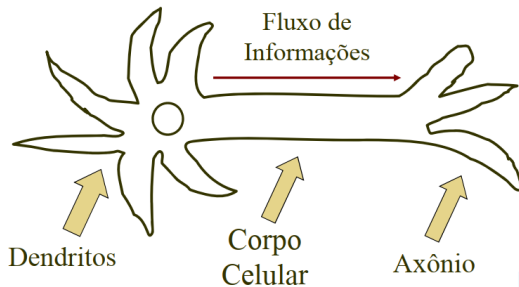
**Disciplina: Redes Neurais Artificiais**

Prof. Dr. Lucas C. Ribas

Programa de Pós-Graduação em Ciência da Computação  
Departamento de Ciências de Computação e Estatística  
Universidade Estadual Paulista (UNESP)



# Representação do Neurônio Biológico



- **Dendritos:** Recebem informações (impulsos nervosos) de outros neurônios.
- **Corpo Celular:** Processa as informações recebidas, integrando-as.
- **Axônio:** Transmite as informações processadas para outros neurônios ou células-alvo.
- O fluxo da informação ocorre dos dendritos para o axônio, de maneira unidirecional.

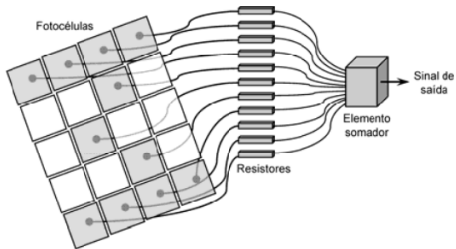
# Rede Perceptron

Aspectos históricos do Perceptron:

- É a **forma mais simples** de configuração de uma rede neural artificial (idealizada por Rosenblatt, 1958).
- Constituída de **apenas uma camada**, contendo **somente um neurônio** nessa única camada.
- Seu propósito inicial era implementar um modelo computacional inspirado na retina, com foco na **percepção eletrônica de sinais**.
- Aplicações iniciais: **identificação de padrões geométricos**.

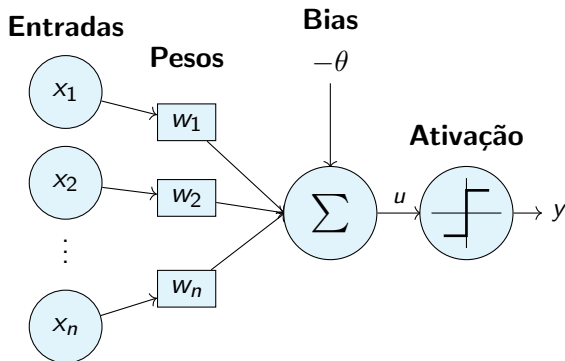
# Rede Perceptron

Modelo ilustrativo do Perceptron para reconhecimento de padrões:



- 1 Sinais elétricos das **fotocélulas** mapeando padrões geométricos são ponderados por **resistores sintonizáveis**.
- 2 Os **resistores** são ajustados durante o processo de treinamento.
- 3 Um **somador** realiza a composição de todos os sinais.
- 4 Como resultado, o **Perceptron** pode reconhecer padrões geométricos, como letras e números.

# Arquitetura do Perceptron



- Sinais de entrada:  $\{x_1, x_2, \dots, x_n\}$
- Pesos sinápticos:  $\{w_1, w_2, \dots, w_n\}$
- Combinador linear:  $z = \sum w_i x_i$
- Limiar de ativação:  $\theta$
- Potencial de ativação:  $u = z - \theta$
- Função de ativação:  $g(u)$
- Sinal de saída:  $y = g(u)$

- 1 Embora seja uma rede simples, o **Perceptron** atraiu pesquisadores interessados na nova área.
- 2 Teve destaque na comunidade de **Inteligência Artificial**.
- 3 É tipicamente usado para **classificação de padrões**.

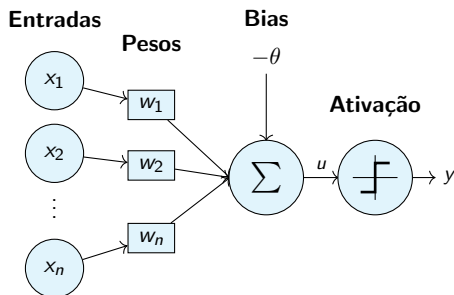
# Princípio de Funcionamento

Passos para obtenção da resposta (saída):

- 1 Apresentação das variáveis de entrada do neurônio.
- 2 Multiplicação de cada entrada  $x_i$  por seu peso sináptico  $w_i$ .
- 3 Cálculo do **potencial de ativação**:

$$u = \sum_{i=1}^n w_i x_i - \theta$$

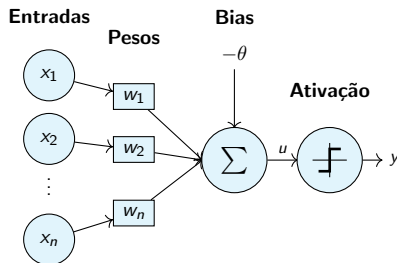
- 4 Aplicação da função de ativação:  $y = g(u)$
- 5 Compilação da saída a partir da aplicação da função de ativação neural em relação ao seu potencial de ativação



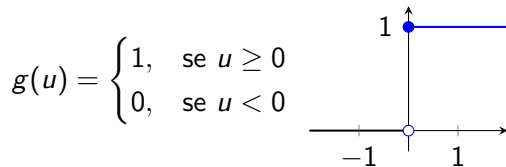
# Princípio de Funcionamento

- Tipicamente, devido as suas características estruturais, o Perceptron usa funções de ativação do tipo: **Degrau** e **Degrau Bipolar**.
- Produzem decisões binárias:  $y \in \{0, 1\}$  ou  $y \in \{-1, 1\}$

$$u = \sum_{i=1}^n w_i x_i - \theta \Rightarrow y = g(u)$$

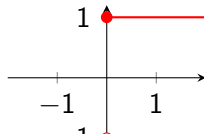


Função Degrau

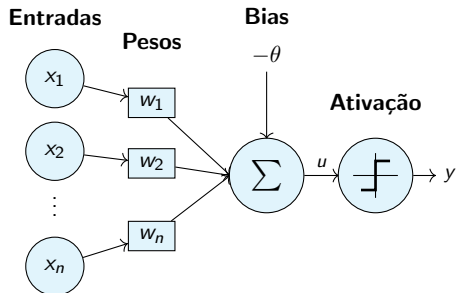


Degrau Bipolar

$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ -1, & \text{se } u < 0 \end{cases}$$



# Princípio de Funcionamento



## Parâmetros do Perceptron

| Parâmetro               | Variável | Tipo Característico      |
|-------------------------|----------|--------------------------|
| Entradas                | $x_i$    | Reais ou Binária         |
| Pesos Sinápticos        | $w_i$    | Reais                    |
| Limiar                  | $\theta$ | Real                     |
| Saída                   | $y$      | Binária                  |
| Função de Ativação      | $g(u)$   | Degrau ou Degrau Bipolar |
| Processo de Treinamento | —        | Supervisionado           |
| Regra de Aprendizado    | —        | Regra de Hebb            |

- 1 Ajuste dos pesos e limiar é efetuado utilizando processo de treinamento **Supervisionado**.
- 2 Para cada amostra dos sinais de entrada se tem a respectiva saída (resposta) desejada.
- 3 Como o *Perceptron* é tipicamente usado em problemas de classificação de padrões, a sua saída pode assumir somente dois valores possíveis.
- 4 Cada um de tais valores será associado a uma das “**duas classes**” que o *Perceptron* identificará.

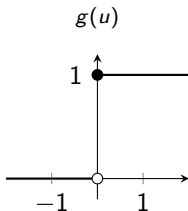


# Aspectos de Aplicabilidade

- 1 Para problemas de classificação, assume-se duas classes possíveis: **Classe A** e **Classe B**.
- 2 A saída do perceptron é associada a cada uma das classes.

## Degrau

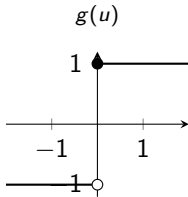
$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ 0, & \text{se } u < 0 \end{cases}$$



$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \Rightarrow x \in \text{Classe A} \\ 0, & \text{se } u < 0 \Rightarrow x \in \text{Classe B} \end{cases}$$

## Degrau Bipolar

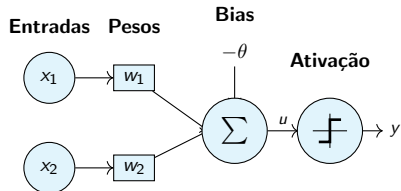
$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ -1, & \text{se } u < 0 \end{cases}$$



$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \Rightarrow x \in \text{Classe A} \\ -1, & \text{se } u < 0 \Rightarrow x \in \text{Classe B} \end{cases}$$

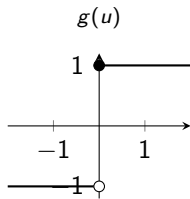
# Análise matemática: O que acontece internamente?

- Para mostrar o que ocorre internamente, assume-se aqui um *Perceptron* com 2 entradas:



$$u = \sum_{i=1}^{n=2} w_i \cdot x_i - \theta \Rightarrow w_1 \cdot x_1 + w_2 \cdot x_2 - \theta$$

Usando como ativação a função sinal, a saída do *Perceptron* será dada por:



$$y = g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ -1, & \text{se } u < 0 \end{cases}$$

$$y = g(u) = \begin{cases} 1, & \text{se } u \geq 0 \Leftrightarrow w_1 x_1 + w_2 x_2 - \theta \geq 0 \\ -1, & \text{se } u < 0 \Leftrightarrow w_1 x_1 + w_2 x_2 - \theta < 0 \end{cases}$$

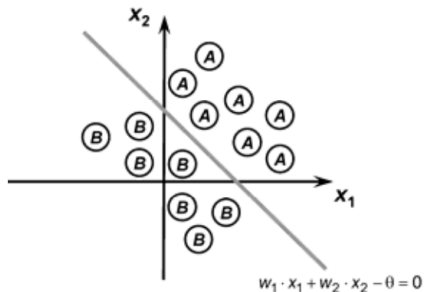
**Conclusão:** O **Perceptron** é um classificador linear que pode ser usado para classificar duas classes **linearmente separáveis**.

# Classificação de padrões usando o Perceptron

- Da conclusão do slide anterior, observam-se que as desigualdades são representadas por uma expressão de primeiro grau (linear).
- A fronteira de decisão para esta instância (*Perceptron* de duas entradas) será então uma reta cuja equação é definida por:

$$w_1 \cdot x_1 + w_2 \cdot x_2 - \theta = 0$$

Representação gráfica da saída do perceptron

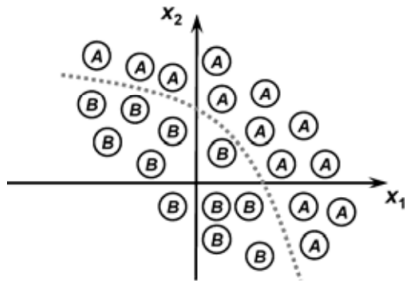
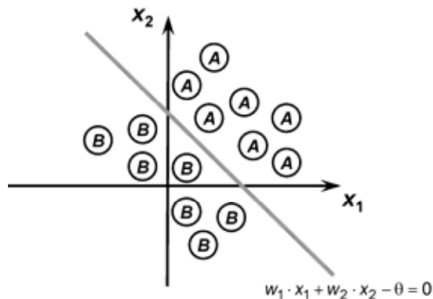


$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \Rightarrow x \in \text{Classe A} \\ -1, & \text{se } u < 0 \Rightarrow x \in \text{Classe B} \end{cases}$$

- Em suma, para a circunstância ao lado, o **Perceptron** consegue então dividir duas classes linearmente separáveis.
- A saída do neurônio com valor **1** significa que os padrões (**Classe A**) estão localizados acima da reta de separação.
- Quando a saída for **-1**, significa que os padrões (**Classe B**) estão abaixo desta fronteira.

# Aspectos de Classificação de Padrões

- Se o **Perceptron** fosse constituído de três entradas (três dimensões), a fronteira de separação seria representada por um **plano**.
- Se o **Perceptron** fosse constituído de quatro ou mais entradas, suas fronteiras seriam **hiperplanos**.
- **Conclusão:** A condição necessária para que o **Perceptron** de camada simples possa ser utilizado como um classificador de padrões é que as classes do problema a ser mapeado sejam **linearmente separáveis**.



# Processo de treinamento: Regra de Hebb

- O ajuste dos pesos  $\{w_i\}$  e limiar  $\{\theta\}$  do *Perceptron*, visando propósitos de classificação binária, é feito por meio da **regra de aprendizado de Hebb**.
- Se a saída produzida pelo *Perceptron* está coincidente com a saída desejada, os seus pesos e limiares são então **mantidos inalterados**.
- Caso a saída seja diferente do valor desejado,  $\{w_i\}$  e  $\{\theta\}$  são então **proporcionalmente ajustados** frente aos valores de suas entradas.
- Este processo é repetido, sequencialmente, para todas as amostras de treinamento, até que a saída produzida pelo *Perceptron* seja similar à saída desejada de cada amostra.

# Processo de treinamento: Regra de Hebb

Em termos matemáticos, tem-se:

$$\begin{cases} w_i^{\text{atual}} = w_i^{\text{anterior}} + \eta \cdot (d^{(k)} - y) \cdot x_i^{(k)} \\ \theta^{\text{atual}} = \theta^{\text{anterior}} + \eta \cdot (d^{(k)} - y) \cdot (-1) \end{cases}$$

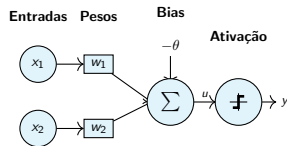
A taxa de aprendizagem  $\eta$  exprime o quão rápido o processo de treinamento da rede estará sendo conduzido rumo à sua convergência.

- $w_i$  são os pesos sinápticos.
- $\theta$  é o limiar do neurônio.
- $x_i^{(k)}$  é o valor da entrada  $i$  da  $k$ -ésima amostra de treinamento.
- $d^{(k)}$  é a saída desejada para a  $k$ -ésima amostra.
- $y$  é a saída do Perceptron.
- $\eta$  é a taxa de aprendizado da rede.

# Intuição da Atualização

A equação de atualização dos pesos do Perceptron é:

$$w_i^{\text{atual}} = w_i^{\text{anterior}} + \eta \cdot (d^{(k)} - y) \cdot x_i^{(k)}$$



A diferença  $d - y$  determina como o peso será ajustado (p/ a função degrau.):

| Situação                      | $d - y$ | Intuição                               |
|-------------------------------|---------|----------------------------------------|
| Acerto                        | 0       | Nada muda: o peso já está correto.     |
| Erro: previu 0, deveria ser 1 | +1      | Aumenta o peso (reforça a entrada).    |
| Erro: previu 1, deveria ser 0 | -1      | Diminui o peso (enfraquece a entrada). |

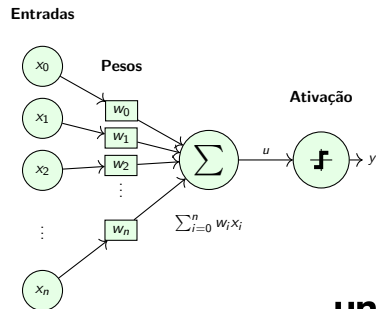
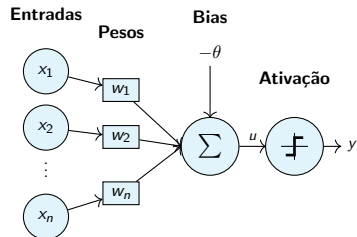
**Resumo:** O Perceptron **reforça o que ajuda a acertar** e **corrige o que leva ao erro**, ajustando os pesos com base na entrada envolvida.

# Aspectos de Implementação Computacional

- Em termos de implementação, torna-se mais conveniente tratar as expressões anteriores em sua forma vetorial.
- Como a mesma regra de ajuste é aplicada tanto para os pesos  $w_i$  como para o limiar  $\theta$ , pode-se então inserir  $\theta$  dentro do vetor de pesos:

$$\mathbf{w} = [\theta \quad w_1 \quad w_2 \dots w_n]^T$$

- O valor do limiar é tratado como uma variável que também será ajustada durante o treinamento.





# Aspectos de Implementação Computacional

- Portanto, tem-se:

$$\begin{cases} w_i^{\text{atual}} = w_i^{\text{anterior}} + \eta \cdot (d^{(k)} - y) \cdot x_i^{(k)} \\ \theta_i^{\text{atual}} = \theta_i^{\text{anterior}} + \eta \cdot (d^{(k)} - y) \cdot x^{(k)} \end{cases}$$

- Notação vetorial:

$$\mathbf{w}^{\text{atual}} = \mathbf{w}^{\text{anterior}} + \eta \cdot (d^{(k)} - y) \cdot \mathbf{x}^{(k)}$$

- Notação algorítmica:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (d^{(k)} - y) \cdot \mathbf{x}^{(k)}$$

- Formato de cada amostra de treinamento:

$$\mathbf{x}^{(k)} = [-1 \quad x_1^{(k)} \quad x_2^{(k)} \quad \dots \quad x_n^{(k)}]^T$$

# Aspectos de Implementação Computacional

Para montagem de conjuntos de treinamento, suponha:

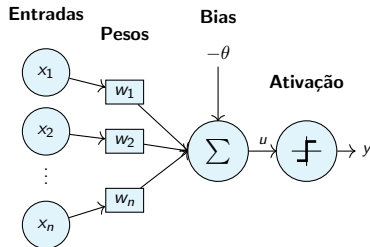
- Um problema a ser mapeado pelo **Perceptron** com três entradas  $\{x_1, x_2, x_3\}$ .
- Quatro amostras com os seguintes valores de entrada e saídas desejadas:

**Amostra 1:** Entrada:  $[0.1, 0.4, 0.7] \Rightarrow$  Saída desejada:  $[1]$

**Amostra 2:** Entrada:  $[0.3, 0.7, 0.2] \Rightarrow$  Saída desejada:  $[-1]$

**Amostra 3:** Entrada:  $[0.6, 0.9, 0.8] \Rightarrow$  Saída desejada:  $[-1]$

**Amostra 4:** Entrada:  $[0.5, 0.7, 0.1] \Rightarrow$  Saída desejada:  $[1]$



# Aspectos de implementação computacional

- Então, pode-se converter tais sinais para que os mesmos possam ser usados no treinamento do **Perceptron**:

## Conjunto de Treinamento:

$$\mathbf{x}^{(1)} = [-1, 0.1, 0.4, 0.7]^T \Rightarrow d^{(1)} = 1$$

$$\mathbf{x}^{(2)} = [-1, 0.3, 0.7, 0.2]^T \Rightarrow d^{(2)} = -1$$

$$\mathbf{x}^{(3)} = [-1, 0.6, 0.9, 0.8]^T \Rightarrow d^{(3)} = -1$$

$$\mathbf{x}^{(4)} = [-1, 0.5, 0.7, 0.1]^T \Rightarrow d^{(4)} = 1$$

$$\Omega(\mathbf{x}) = \begin{matrix} & \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \mathbf{x}^{(3)} & \mathbf{x}^{(4)} \\ \begin{matrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{matrix} & \begin{bmatrix} -1 \\ 0.1 \\ 0.4 \\ 0.7 \end{bmatrix} & \begin{bmatrix} -1 \\ 0.3 \\ 0.7 \\ 0.2 \end{bmatrix} & \begin{bmatrix} -1 \\ 0.6 \\ 0.9 \\ 0.8 \end{bmatrix} & \begin{bmatrix} -1 \\ 0.5 \\ 0.7 \\ 0.1 \end{bmatrix} \end{matrix}$$
$$\Omega(\mathbf{d}) = \begin{bmatrix} d^{(1)} & d^{(2)} & d^{(3)} & d^{(4)} \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

- Forma Matricial:** as amostras de treinamento são organizadas como colunas em uma matriz:

# Pseudocódigo para fase de treinamento

## Início {Algoritmo *Perceptron* – Fase de Treinamento}

```
<1> Obter o conjunto de amostras de treinamento  $\{ \mathbf{x}^{(k)} \}$ ;  
<2> Associar a saída desejada  $\{ d^{(k)} \}$  para cada amostra obtida;  
<3> Iniciar o vetor  $\mathbf{w}$  com valores aleatórios pequenos;  
<4> Especificar a taxa de aprendizagem  $\{\eta\}$ ;  
<5> Iniciar o contador de número de épocas  $\{época \leftarrow 0\}$ ;  
<6> Repetir as instruções:  
    {  
        <6.1> erro  $\leftarrow$  "inexiste";  
        <6.2> Para todas as amostras de treinamento  $\{ \mathbf{x}^{(k)}, d^{(k)} \}$ , fazer:  
            {  
                <6.2.1>  $u \leftarrow \mathbf{w}^T \cdot \mathbf{x}^{(k)}$ ;  
                <6.2.2>  $y \leftarrow \text{sinal}(u)$ ;  
                <6.2.3> Se  $y \neq d^{(k)}$   
                    <6.2.3.1> Então  $\begin{cases} \mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (d^{(k)} - y) \cdot \mathbf{x}^{(k)} \\ \text{erro} \leftarrow \text{"existe"} \end{cases}$   
            }  
        <6.3>  $época \leftarrow época + 1$ ;  
    }  
    Até que: erro = "inexiste"
```

Fim {Algoritmo *Perceptron* – Fase de Treinamento}

## Época de treinamento:

Cada apresentação completa de todas as amostras pertencentes ao subconjunto de treinamento, visando o ajuste dos pesos sinápticos e limiares dos seus neurônios.

# Pseudocódigo para fase de teste

## Início {Algoritmo *Perceptron* – Fase de Operação}

<1> Obter uma amostra a ser classificada  $\{ \mathbf{x} \}$ ;  
<2> Utilizar o vetor  $\mathbf{w}$  ajustado durante o treinamento;  
<3> Executar as seguintes instruções:  
    <3.1>  $u \leftarrow \mathbf{w}^T \cdot \mathbf{x}$  ;  
    <3.2>  $y \leftarrow \text{sinal}(u)$  ;  
    <3.3> Se  $y = -1$   
        <3.3.1> Então: amostra  $\mathbf{x} \in \{\text{Classe A}\}$   
    <3.4> Se  $y = 1$   
        <3.4.1> Então: amostra  $\mathbf{x} \in \{\text{Classe B}\}$

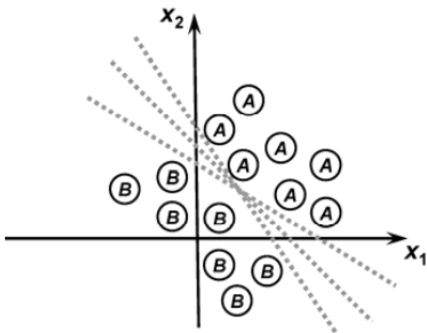
## Fim {Algoritmo *Perceptron* – Fase de Operação}

**Obs 1:** A *Fase de Operação ou Teste* é usada somente após a fase de treinamento, pois aqui a rede já está apta para ser usada no processo.

**Obs 2:** A *Fase de Operação ou Teste* é então utilizada para realizar a tarefa de classificação frente às novas amostras que serão apresentadas em suas entradas.

# Ilustração do processo de convergência

O processo de treinamento tende a mover continuamente o hiperplano de classificação até que seja alcançada uma **fronteira de separação** que permita dividir as duas classes. Como exemplo, considere uma rede com apenas duas entradas  $\{x_1, x_2\}$ .



- Após a primeira época, o hiperplano ainda está distante da separação ideal.
- A distância tende a diminuir com o avanço das épocas de treinamento.
- Quando o **Perceptron** converge, indica que a fronteira foi finalmente alcançada.
- A reta de separabilidade a ser produzida após o treinamento do Perceptron não é única.
- O número de épocas pode também variar de treinamento para treinamento.

# Reflexões, observações e aspectos práticos

- ❶ O que acontece com o algoritmo do **Perceptron** se o problema for **não-linearmente separável**?
- ❷ Como tratar a limitação anterior no **Perceptron**?
- ❸ Quais são os inconvenientes de usar taxas de aprendizagem muito grandes ou muito pequenas?
- ❹ Dois projetistas aplicam o mesmo problema no **Perceptron**. O vetor de pesos final será o mesmo?

# Referência

Slides baseados em:

- Notas de aula AULA 03 – Redes Perceptron, disciplina de "Redes Neurais Artificiais". Prof. Ivan Nunes da Silva.
- Silva, Ivan Nunes da, Danilo Hernane Spatti, and Rogério Andrade Flauzino. "Redes neurais artificiais para engenharia e ciências aplicadas." (2010).